

ICS 613: Software Engineering

Week 02 – Requirements Engineering

Information and Computer Sciences
University of Hawaii at Manoa

What is requirements engineering?

The process of discovering, documenting, and managing what a system should do

Elicitation	Specification	Validation	Management
Discover what stakeholders need through interviews, workshops, observation, and prototyping	Document requirements as user stories, use cases, and acceptance criteria that all parties agree on	Confirm that requirements are correct, complete, consistent, and actually achievable	Track changes, maintain priorities, and keep traceability as scope evolves throughout the project

Three kinds of requirements

- **Functional**: Defines the behavior of the system – what the system **should do**
 - e.g.: The home page of the university should show the five most recent news items
- **Non-Functional**: These are **quality attributes** on how the system will perform its desired tasks – how **well** the system does it
 - e.g.: The home page of the university should load in under 1 second
- **Constraints**: **Limitations** imposed on the system's design or implementation choices, such as technology, budget, regulations, or platform
 - e.g.: The system must be built using Java and deployed on the university's infrastructure

Software quality attributes

- These non-functional requirements are expressed as a system's quality attributes
- Not all systems have the same quality attributes
- Determining/eliciting the quality attributes of a systems is usually done by the architect/tech lead/BA
- They form an essential part of planning/designing the architecture of the system
- *Informally, they are known as “ilities”*

Types of quality attributes

- **Accessibility** – support individuals with disabilities
- **Internationalization/Localization** – support different languages/regions
- **Maintainability** - ease with which the system can be maintained (e.g., bug fix, requirement updates, etc.)
- **Portability** – extent to which the s/w system runs on multiple platforms
- **Testability** – extent to which s/w supports testing
- **Usability** – Effective and efficient use of the s/w system by users
- ...

Quality conflicts

- It is not possible for any system to be optimized for all quality attributes – example:
 - improving robustness may lead to loss of performance
 - encrypting/decrypting secured data leads to performance issues (time & resources)
 - due to budget and resource constraints, it is not feasible to support users with all forms of disabilities
 - Supporting older web browsers (e.g., IE 6) will consume time & money

Requirement Elicitation

Elicitation techniques

Elicitation is the process of discovering requirements from stakeholders and other sources

Interviews & Surveys

Unstructured:

- One-on-one conversations
- Exploratory; good for early discovery

Structured:

- Fixed questions
- Comparable across stakeholders

Workshops

- Group sessions with multiple/key stakeholders (e.g., business, IT...)
- Whiteboards and sticky notes to capture ideas visually
- Everyone in the room should contribute
- Resolve conflicts in the room

Observation

- Watch users in their real environment
- Reveals unstated behaviors and informal processes
- Combine with interviews afterwards for clarity and confirmations

Document Analysis

Review existing forms, reports, policies, and legacy documents to understand the current process, data, and business rules in use

Prototyping

Mockups to surface requirements users can't easily articulate:

- Low-fidelity: Paper sketches; digital wireframes
- High fidelity: interactive prototypes

Risk: stakeholders mistake a prototype for finished code

No single technique is sufficient – **use a combination** for comprehensive coverage

Stakeholder identification

- Identify the stakeholders **before eliciting** requirements
 - New stakeholders may also emerge during the elicitation process
- A stakeholder is anyone who is **affected by or can influence** the system
 - Examples: end users, customers, developers, managers, regulators, operations staff
- Identifying the right stakeholders early prevents missing critical requirements

Requirement Specification

User stories

- A user story is a short, **user-centric** description of a requirement
- Format: “**As a [role], I want [goal] so that [benefit]**”
 - e.g.: “As a student, I want to view my grades online so that I can track my progress”
- **Acceptance criteria** define when a story is complete (i.e., testable conditions)
 - e.g.: “System shows the students letter grade plus percentage”
- The “so that” clause is not optional: it tells the team **why the feature matters** and helps prevent unnecessary features.

Acceptance Criteria

Every user story needs acceptance criteria

- Specific, verifiable conditions that **must be true** for the story to be complete
- Removes ambiguity – client, dev, QA all **agree on what 'done' means**
- Each scenario maps to at least one automated or manual **test**
- Forces **edge-case** thinking before any implementation begins
- **Prevents scope creep** by identifying out-of-scope requests early
- Facilitates code reviews as **reviewers verify behavior**

What to include:

- Specific behavior
- Observable result
- Normal path
- Edge/error paths
- Permission rules

Note: Refer course project document for examples

INVEST criteria for user stories

A mnemonic for writing high-quality user stories that are ready for sprint planning

Letter	Criterion	What It Means	Why It Matters
I	Independent	The story can be developed and delivered on its own, without depending on other stories	<i>Enables flexible prioritization and parallel work across teams</i>
N	Negotiable	Details are open to discussion; the story is a placeholder for conversation, not a rigid contract	<i>Encourages collaboration between developers and stakeholders</i>
V	Valuable	The story delivers clear value to the user or customer, not just technical tasks	<i>Keeps the team focused on outcomes that matter to the business</i>
E	Estimable	The team can reasonably estimate the effort required to implement the story	<i>Supports accurate sprint planning and capacity allocation</i>
S	Small	The story is small enough to be completed within a single sprint iteration	<i>Reduces risk and enables faster feedback loops</i>
T	Testable	Clear acceptance criteria exist so the team knows when the story is done	<i>Ensures quality and provides a shared definition of complete</i>

Domain modeling

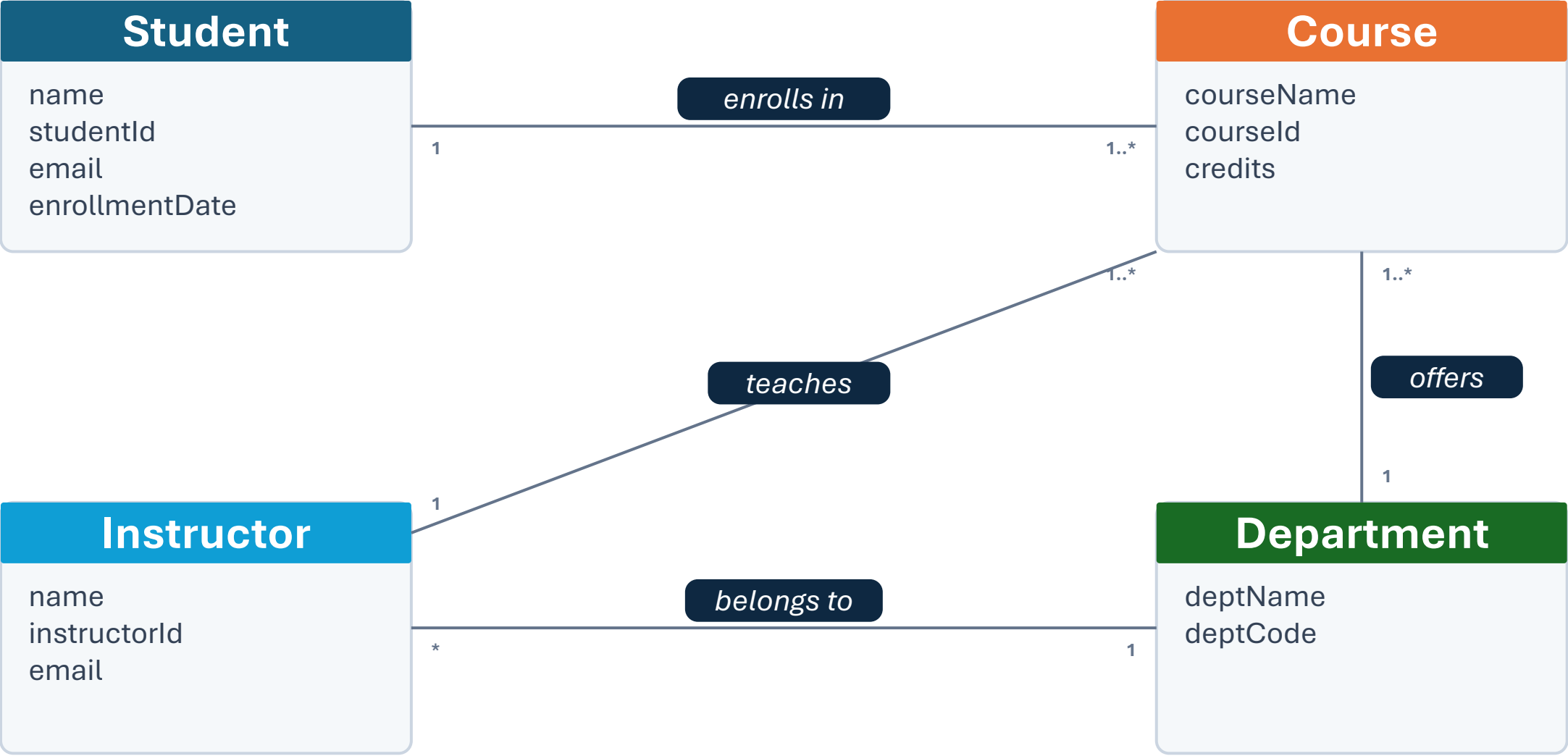
- A domain model is a **conceptual representation** of the key entities, attributes, and relationships in the problem domain
- It describes **what** exists in the real world the system must support (*not how the software will be built*)
- Purpose of domain modeling
 - Builds a shared understanding between stakeholders and developers
 - Bridges the gap between requirements and design
 - Identifies key concepts before any code is written
 - Reduces ambiguity and uncovers hidden assumptions early
- Domain models are part of **Object-Oriented Analysis (OOA)** and use UML class diagram notation

The domain model is not a database schema. It can be similar to the schema, but they are different artifacts

Entities, attributes, and relationships

- **Entities**: the important **nouns** in the problem domain (i.e., real-world objects or concepts)
 - e.g.: Student, Course, Instructor, Department
- **Attributes**: descriptive **properties of an entity**
 - e.g.: Student has name, studentId, email, enrollmentDate
 - In a domain model, attributes should not have visibility modifiers or types (those are design decisions)
- **Relationships**: meaningful **associations between entities**, often derived from verbs
 - e.g.: a student *enrolls* in a course, an instructor *teaches* a course
- **Multiplicity**: every relationship should be named and have multiplicity (cardinality)
 - e.g.: a student enrolls in one or more courses ← one-to-many (1:N)

University domain model



Writing good requirements

A well-written requirement has these qualities:

- **Correct**: accurately reflects a real stakeholder need
- **Unambiguous**: open to only one interpretation
- **Complete**: contains all necessary information to implement and test
- **Consistent**: does not contradict any other requirement
- **Verifiable**: can be tested or demonstrated to confirm it is met
- **Traceable**: can be linked back to its origin and forward to design/test
 - Bad: “The system should be fast”
 - Good: “The system shall respond to user input within 2 seconds”

The SRS document

- SRS = **Software Requirements Specification** – the central requirements document
- **IEEE 830 / ISO/IEC/IEEE 29148** define the standard structure
- The SRS describes **what** the system must do, **not how** it will be built

<https://ieeexplore-ieee-org.eres.library.manoa.hawaii.edu/document/6146379>

Requirement Validation

Requirements validation

- Validation: “Are we building the right product?”
- Verification: “Are we building the product right?”
- Validation checks that requirements define the system the customer actually wants
- Key validation techniques
 - Requirements reviews: stakeholders inspect the requirement artifacts for errors and gaps
 - Prototyping: users interact with a mock-up to confirm expectations
 - Test case generation: if you cannot write a test for a requirement, it is likely flawed
 - Walk-throughs: informal team discussions to assess feasibility
- Catching errors early in requirements costs significantly less than fixing them in production

Requirements traceability

- Traceability **links each requirement** to its origin and to downstream artifacts
- A **Requirements Traceability Matrix** maps requirements → design → code → tests
- **Forward** traceability: requirement → design → implementation → test
 - Ensures every requirement is implemented and tested
- **Backward** traceability: test → implementation → design → requirement
 - Ensures every feature traces to an approved requirement
- Traceability helps with impact analysis when requirements change
- Essential for regulated industries (medical, aerospace, automotive)

Managing Requirements

The product backlog

- A **prioritized list** of everything the team might build
 - User stories, bug fixes, technical debt, spikes/research, and non-functional requirements
- **Single source of truth:** all features, enhancements, and fixes live in one place, eliminating scattered requirements
- **Drives sprint planning:** the team pulls items from the top each sprint, ensuring the most valuable work is always next
- **Enables adaptability:** unlike a fixed specification, the backlog evolves as the team learns from feedback and changing needs
- Items near the top should be **small, well-defined, and estimated;** items lower down can remain as rough ideas

Managing Requirements with GitHub

- GitHub Issues as user stories
 - Each user story becomes a GitHub Issue
 - Issues are tracked, assigned, linked to code, and closed when the story is done
- GitHub Projects to visualize the backlog
 - Kanban view
 - Cards move as status change
- Utilize custom Labels for filtering
 - Priority: high, medium, low
 - Type: story, bug, task, etc.
 - Actor: member, owner, etc.
- Milestones

16 Task Backlog

Added by ChrisChV

Implementar las interfaces para las alertas de cercacia para obtener una promoción
#299 opened by ChrisChV in ChrisChV/Nogal
nogal task

Added by ChrisChV

Implementar el frontend para mostrar el listado de productos de regalo
#301 opened by ChrisChV in ChrisChV/Nogal
nogal task

Added by ChrisChV

Implementar el backend para generar formularios en excel con diferentes filtros
#13 opened by ChrisChV in ChrisChV/mine_supervision
mina task

Added by ChrisChV

Implementar el backend para generar un formulario en pdf
#12 opened by ChrisChV in ChrisChV/mine_supervision
mina task

Added by ChrisChV

12 To do

Added by ChrisChV

Integrar el backend para obtener el progreso del mes anterior con las interfaces
#280 opened by ChrisChV in ChrisChV/Nogal
nogal task

Added by ChrisChV

Implementar el backend para la funcionalidad de buscar promociones
#294 opened by ChrisChV in ChrisChV/Nogal
nogal task

Added by ChrisChV

Implementa el backend para mostrar las alertas de cercanía para obtener una promoción
#298 opened by ChrisChV in ChrisChV/Nogal
nogal task

Added by ChrisChV

Implementar la lógica para llenar un formulario
#10 opened by ChrisChV in ChrisChV/mine_supervision
mina task

2 In progress

Added by ChrisChV

Diseñar el prototipo de alta fidelidad del BizzyEdx
#246 opened by ChrisChV in ChrisChV/BizzyChain
bizzychain task

Added by ChrisChV

Diseñar el prototipo de alta fidelidad del BizzySearch
#245 opened by ChrisChV in ChrisChV/BizzyChain
bizzychain task

4 In review

Added by ChrisChV

Agregar en el diseño la funcionalidad para agregar productos en cantidades predeterminadas
#281 opened by ChrisChV in ChrisChV/Nogal
nogal task

Added by ChrisChV

Crear modelos y configurar lugares con los supervisores
#7 opened by ChrisChV in ChrisChV/mine_supervision
mina task

Added by ChrisChV

Implementar el backend para detectar las promociones y agregar los premios
#297 opened by ChrisChV in ChrisChV/Nogal
nogal task

Added by ChrisChV

Implementar el backend del listado de productos de regalo por promociones
#300 opened by ChrisChV in ChrisChV/Nogal
nogal task

223 Done

Added by ChrisChV

Implementar el fidelidad del fe
#238 opened by C ChrisChV/BizzyCh
bizzychain tas

Added by ChrisChV

Implement the products to the
#230 opened by C ChrisChV/Nogal
nogal task

Added by ChrisChV

Implementar el b una cuenta banc
Extras#35 opened b
growi task

Added by ChrisChV

Implementar el b ordenamiento m
Extras#33 opened b
growi task

Added by ChrisChV

Implementar el b acumulado en ga
Extras#31 opened b
growi task

Added by ChrisChV

Implementar el b deslogueo
Extras#37 opened b

Agile vs traditional requirements

- **Traditional (Waterfall)**: define all requirements upfront before design begins
 - Comprehensive SRS document, formal change control process
 - Best for: stable domains, regulatory environments, fixed-scope contracts
- **Agile**: requirements emerge iteratively through collaboration with the customer
 - Product backlog of user stories, refined each sprint
 - Best for: evolving domains, frequent feedback, small-to-medium teams
- Key difference: Agile **embraces change**; traditional **controls it**
- Many teams use a hybrid: high-level requirements upfront, detailed stories just-in-time

Common pitfalls in requirements engineering

- **Ambiguous requirements:** vague terms like “user-friendly” or “fast” cause misinterpretation
- **Poor communication:** Can lead to missing or incorrectly implemented requirements
- **Scope creep:** uncontrolled addition of new requirements without adjusting schedule or budget
- **Gold plating:** developers add unrequested features, wasting resources
- **Missing stakeholders:** failing to identify key stakeholders leads to gaps in requirements
- **Confusing requirements with solutions:** “use a dropdown menu” is a solution, not a requirement
- **Insufficient validation:** skipping reviews means errors propagate to design and code

Further reading

- SWEBOK Software Requirements Knowledge Area: <https://www.computer.org/education/bodies-of-knowledge/software-engineering/v4>
- User Stories: <https://agilealliance.org/glossary/user-stories>
- Domain Modeling: <https://www.thoughtworks.com/en-us/insights/blog/agile-project-management/domain-modeling-what-you-need-to-know-before-coding>
- UML Distilled: https://uhawaii-manoa.primo.exlibrisgroup.com/permalink/01UHAWAIL_MANOA/c2k370/alma9917993594605682

Questions?

